# CASE STUDY 1

## N Queen Problem with Backtracking

N Queens problem is one of the most common examples of backtracking.

***Goal:*** To place N Queens on an N×N chessboard in such a way that no two queens attack each other.

***Input:*** The size of a chess board. Generally, it is 8 as a normal chess board has size = 8 X 8.

***Output:*** A binary matrix that has 1s for the blocks where queens are placed. For example, the 4 X 4 matrix given below has 1 for positions, where 4 Queens can be safely placed - no queen attacking the other one.

*{0, 1, 0, 0}*

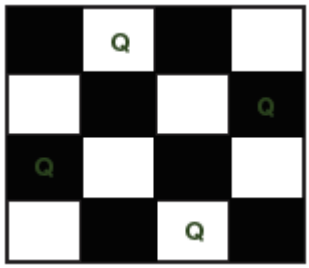
*{0, 0, 0, 1}*

*{1, 0, 0, 0}*

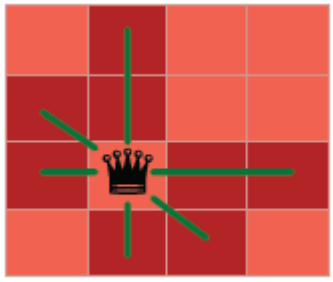
*{0, 0, 1, 0}*

*Note that* the matrix represents in which row and column the N Queens can be placed. The 0s in the matrix denotes the blank spaces on the chess board. If the solution does not exist, the N Queens algorithm should return false.

Chessboard corresponding to the output matrix



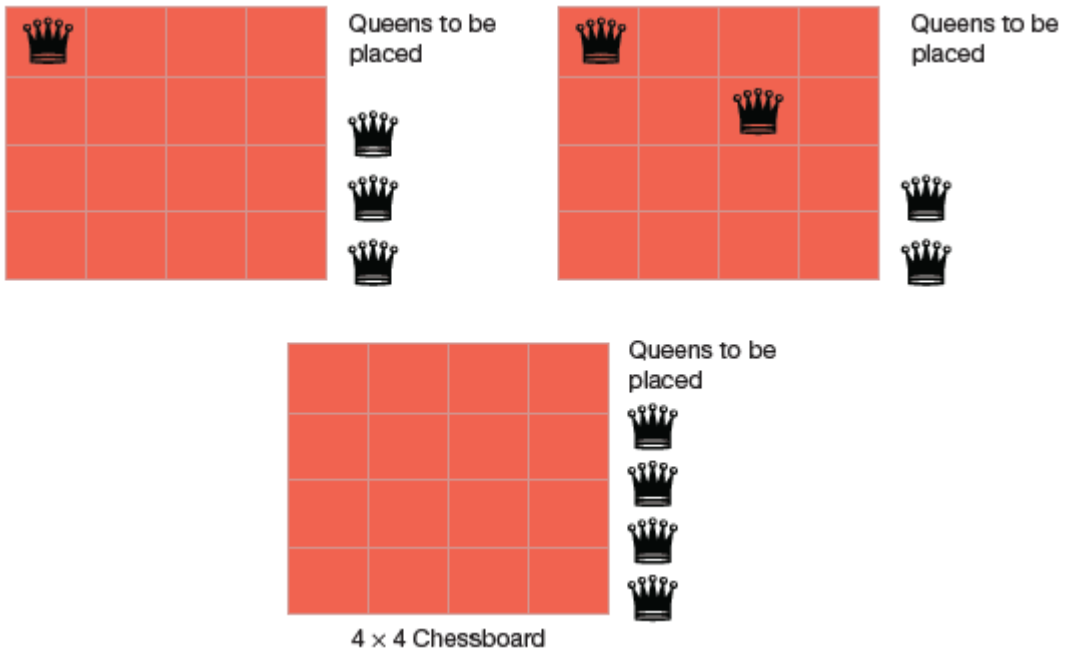

### *Understanding Dynamics of a Chessboard*



Cells attacked by the queen

We start with a 4x4 chessboard and 4 queens that have to be safely placed on it. Let's place the first queen in the first row.

Now, let's place the second queen in a safe position. We cannot place it in the first row, so put the queen in the second row.

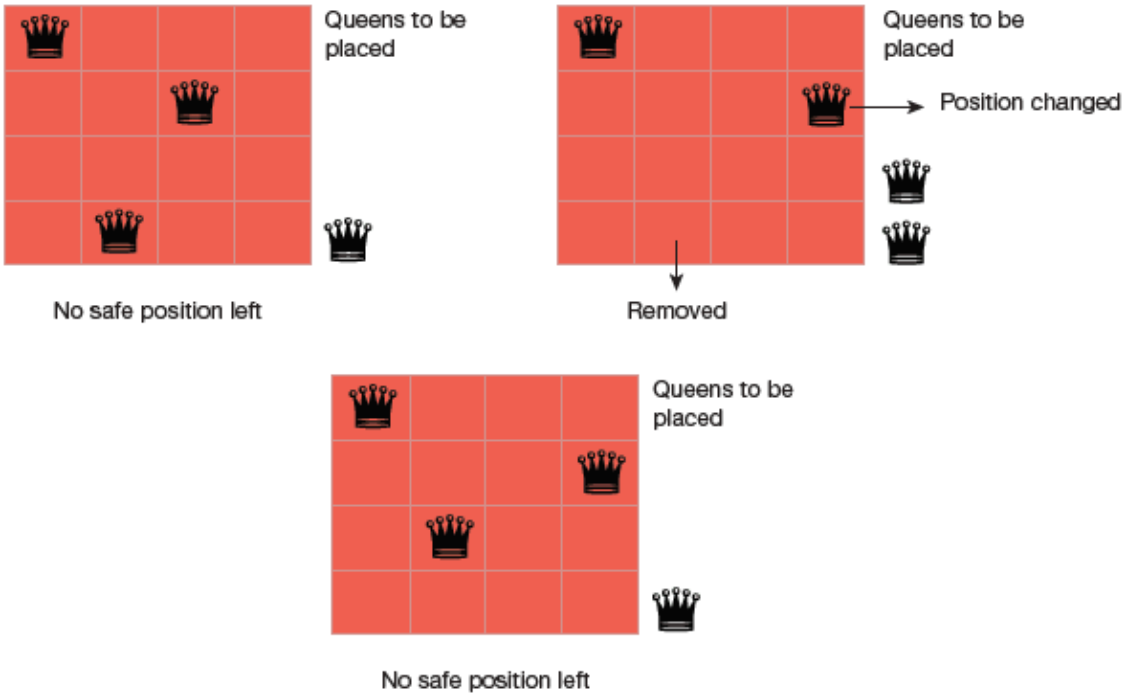



4 × 4 Chessboard

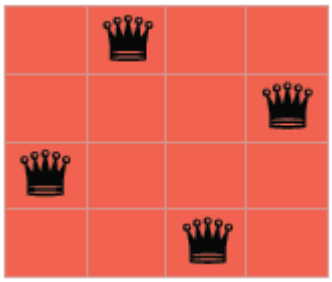
Let's place the third queen in a safe position, somewhere in the third row.

Now, no safe position is left for the last queen. So, we need to change the position of the previous queen i.e., backtrack and change the previous decision.

At this point in time, we see that there is no other position where we can place the third queen, so we need to go back one more step and change the position of the second queen as shown below. Once the second queen is repositioned, the third queen is placed in a safe position in the third row.




Finally, we will place the fourth (the last) queen safely in the fourth row to get the solution as shown below.



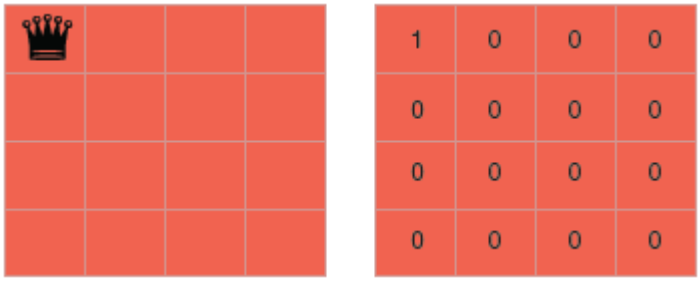
***Algorithm:*** Generate all possible configurations of queens on board and print a configuration that satisfies the given constraints.

```
while there are untried configurations
   generate the next configuration
   if queens don't attack in this configuration, then
       print this configuration
```

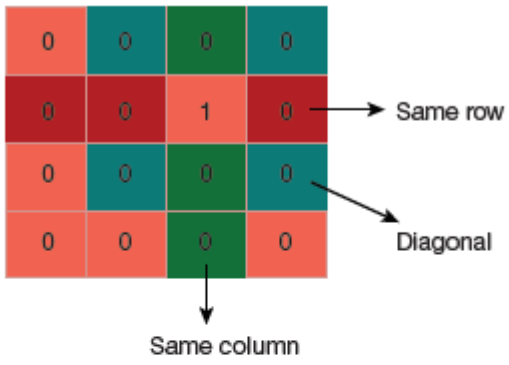
***Domain Knowledge:*** On a chessboard, queens can attack in any direction- horizontal, vertical, and diagonal way.

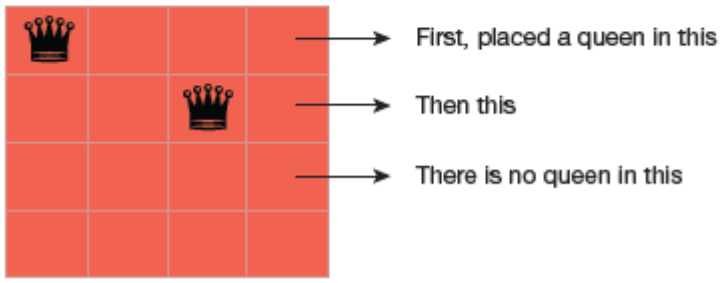
### Code for N Queens

Let us check if a particular position - cell(i,j) is safe to put a queen or not. If a queen already exists in a cell, it has an entry 1 else 0.




The cell (i,j) is prone to attack in three conditions - if there is any other queen in row i, column j or in the diagonals.

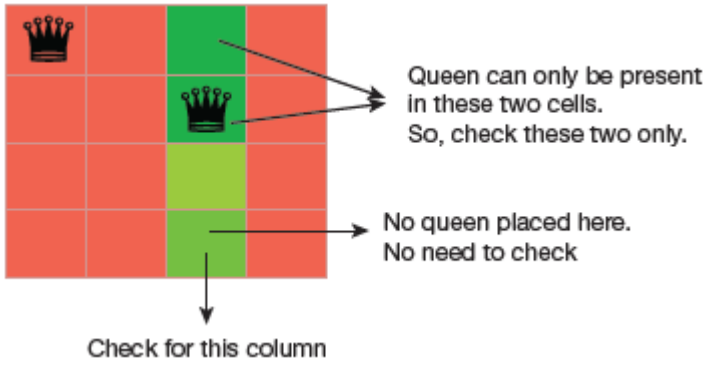



In our algorithm, we start from the first row, placing a queen there and then moving to the next row. So, at any point in time, all the rows above

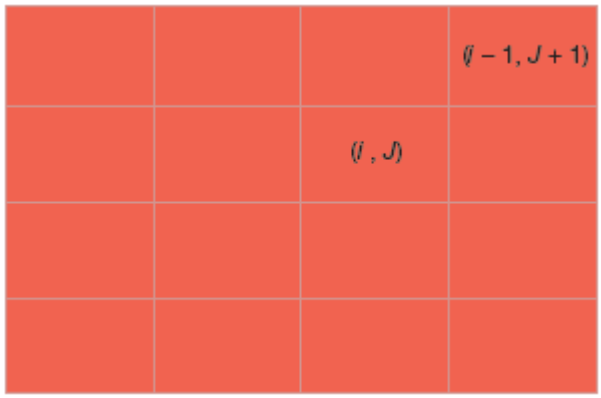
the current row (i) already have a queen, and thus, there is no need to check for rows above the current row.

Next, we can check for the column j by changing k from 1 to i-1 in **`board[k][j]`** because only the rows from 1 to i-1 are filled.



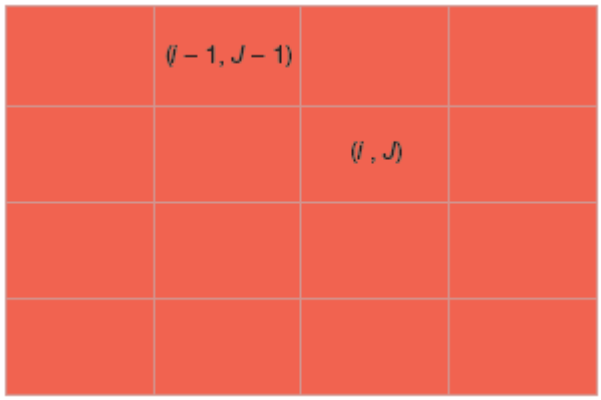

```
for k in 1 to i-1
  if board[k][j]==1
    return TRUE
```

Now, check for the diagonal. We know that all the rows below the row i are empty, so we just have to check the diagonal elements that are above the row i. This can be done by decreasing the value of i and increasing the value of j to traverse the right diagonal elements of any cell(i,j).




```
k = i-1
l = j+1
while k>=1 and l<=N
  if board[k][l] == 1
    return TRUE
  k=k-1
  l=l+1
```

Similarly, we can decrement both the values of i and j of cell (i, j) to traverse over the left diagonal, above the row i.




```
k = i-1
l = j-1
while k>=1 and l>=1
  if board[k][l] == 1
    return TRUE
  k=k-1
  l=l-1
```

Finally return False so that if no case returns a True, then a False is returned indicating the position, cell (i,j) is safe.

We can write the entire code as:

```
IS-ATTACK(i, j, board, N)
  // checking in the column j
  for k in 1 to i-1
    if board[k][j]==1
      return TRUE
  // checking upper right diagonal
  k = i-1
  l = j+1
  while k>=1 and l<=N
    if board[k][l] == 1
      return TRUE
    k=k+1
    l=l+1
  // checking upper left diagonal
  k = i-1
  l = j-1
  while k>=1 and l>=1
    if board[k][l] == 1
      return TRUE
    k=k-1
    l=l-1
  return FALSE
```

Let us now write the code involving backtracking to solve the N Queen problem. The function accepts the row, number of queens, size of the board and the board itself - **`N-QUEEN(row, n, N, board)`** as input.

If the number of queens is 0, then we have already placed all the queens.

```
if n==0
  return TRUE
```

Otherwise, we will iterate over each cell of the board in the row passed to check if we can place the queen in that cell or not. Note that we cannot place the queen in a cell if it is under attack.

```
for j in 1 to N
  if !IS-ATTACK(row, j, board, N)
    board[row][j] = 1
```

After placing the queen in the cell, we will also check if we are able to place the next queen with this arrangement or not. If not, then we will have to backtrack to select a different position for the current queen.

```
        for j in 1 to N
          ...

    if N-QUEEN(row+1, n-1, N, board)
      return TRUE
    board[row][j] = 0
```

If **`N-QUEEN(row+1, n-1, N, board)`** is called to place rest of the queens with the current arrangement. Also, since all the rows up to 'row' are occupied, we now start from 'row+1'. If this returns True, then we are successful in placing all the queen, if not, then we have to change the position of our current queen. So, we vacate the current cell by writing **`board[row][j] = 0`** and then iterating again to find another place for the queen (backtracking).

The base condition of our algorithm is **`- if n==0 → return TRUE`**. Therefore, when all the queens are placed correctly, then **`N-QUEEN(row, 0, N, board)`** will be called and True will be returned. However, if True is not returned then it implies that solution was not found and this False is returned.

```
N-QUEEN(row, n, N, board)
...
  return FALSE
N-QUEEN(row, n, N, board)
  if n==0
    return TRUE
  for j in 1 to N
    if !IS-ATTACK(row, j, board, N)
      board[row][j] = 1
      if N-QUEEN(row+1, n-1, N, board)
        return TRUE
      board[row][j] = 0 //backtracking, changing current decision
  return FALSE
```

## PROGRAM CODE

```
def is_attack(i, j, board, N):
  # checking for column j
  for k in range(1, i):
    if(board[k][j] == 1):
      return True
  # checking upper right diagonal
  k = i-1
  l = j+1
  while (k>=1 and l<=N):
    if (board[k][l] == 1):
      return True
    k=k+1
    l=l+1
  # checking upper left diagonal
  k = i-1
  l = j-1
  while (k>=1 and l>=1):
    if (board[k][l] == 1):
      return True
    k=k-1
    l=l-1
  return False
def n_queen(row, n, N, board):
  if (n==0):
    return True
  for j in range(1, N+1):
    if(not(is_attack(row, j, board, N))):
      board[row][j] = 1
      if (n_queen(row+1, n-1, N, board)):
        return True
      board[row][j] = 0 #backtracking
  return False
if __name__ == '__main__':
  board = [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[0,0,0,
,0]]
  n_queen(1, 4, 4, board)
  #printing the matix
  for i in range(1, 5):
      print(board[i][1:])
```

## OUTPUT

[0, 1, 0, 0]

[0, 0, 0, 1]

[1, 0, 0, 0]

[0, 0, 1, 0]